

Token-Picker: Accelerating Attention in Text Generation with Minimized Memory Transfer via Probability Estimation

Junyoung Park^{1,2}, Myeonggu Kang¹, Yunki Han¹, Yanggon Kim², Jaekang Shin^{1,2}, Lee-Sup Kim¹

¹Korea Advanced Institute of Science and Technology, ²System LSI, Samsung Electronics

{wnsdud9731, jaekangshin}@gmail.com, {mgkang95, yunki.han, leesup}@kaist.ac.kr, yanggon.kim@samsung.com

ABSTRACT

The attention mechanism in text generation is memory-bounded due to its sequential characteristics. Therefore, off-chip memory accesses should be minimized for faster execution. Although previous methods addressed this by pruning unimportant tokens, they fall short in selectively removing tokens with near-zero attention probabilities in each instance. Our method estimates the probability before the softmax function, effectively removing low probability tokens and achieving an 12.1x pruning ratio without fine-tuning. Additionally, we present a hardware design supporting seamless on-demand off-chip access. Our approach shows 2.6x reduced memory accesses, leading to an average 2.3x speedup and a 2.4x energy efficiency.

1 INTRODUCTION

Text generation using Large Language Models [4, 6, 11] have been instrumental in advancing applications such as chatbot systems and virtual assistants. With their growing importance, several companies are striving to integrate these applications into their hosted services, highlighting the importance of efficient inference.

Language models are based on the autoregressive transformer model specialized for text generation. At its core, there is an attention mechanism and a fully connected (FC) layer. The attention mechanism captures context within text sequences, while the FC layer uses weights to transform activations into higher representations. This setup sequentially creates words using prior ones to form complete sentences. However, this sequential property in text generation makes the workload memory-bound, not fully utilizing the computing resources of GPUs. To address this, recent research [5, 9, 13] has enhanced throughput of generation inference by adopting dynamic batching, allowing multiple requests to share the weights. It enables to amortize the transferring cost and improves parallel processing. Still, the attention mechanism faces with memory challenges because each user's sequence remains separate. As more users are batched together, memory accesses demands rise. Thus, minimizing the memory transfer of attention can improve throughput and energy efficiency in batching scenario.

Previous works [1, 8, 15] could achieve this by leveraging softmax redundancy, which turns token correlation scores into probabilities and often produces many near-zero values. However, these approaches overlooked the varying number of unimportant tokens across instances and could not selectively spare the important ones only. Furthermore, retraining is often required for high pruning rate, as the distribution needs to be adjusted to fit each method.

To tackle the aforementioned problems, we introduce an adaptive token pruning method that aligns with each instance. Before completing all correlation calculations, our method estimates token probabilities and eliminates those below a set threshold. Memory accesses during the estimation is minimized by: 1) beginning with the initial bit chunk of a key vector, where a bit chunk refers to a segment of bits from the vector elements; 2) estimating the probability to decide on pruning or retrieving the next chunk; and 3) if the chunk is needed, efficiently managing the access delay by executing independent computation. Notably, our method achieves a higher pruning ratio compared to previous techniques, without the need for retraining.

The **major contributions** of this paper are as follows:

- We propose a probability estimation method to prune redundant tokens, showing 12.1× pruning ratio in average.
- We present out-of-order score calculations to support on-demand DRAM request, further reducing 1.39× transfers.
- We design a tailored hardware to support the proposed method, achieving 2.28× speedup and 2.41× energy efficiency.

2 BACKGROUND & MOTIVATION

2.1 Autoregressive transformer model

2.1.1 Transformer architecture. Language models utilize transformer architecture. Its main components are the self-attention and the feed-forward network (FFN). Self-attention discerns correlations within token embeddings $(\mathbf{x}_1, \dots, \mathbf{x}_n) \in \mathbb{R}^{n \times d}$ to capture context information. The specific operations within the self-attention are detailed in the following equations:

$$\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t = \mathbf{W}_q \mathbf{x}_t, \mathbf{W}_k \mathbf{x}_t, \mathbf{W}_v \mathbf{x}_t \quad (1)$$

$$s_{ti} = \mathbf{q}_t^\top \cdot \mathbf{k}_i / \sqrt{d_h}, \quad p_{ti} = \frac{\exp(s_{ti})}{\sum_{k=1}^t \exp(s_{tk})} \quad \text{for } 1 \leq i \leq t \quad (2)$$

$$\mathbf{o}_t = \sum_{i=1}^t p_{ti} \mathbf{v}_i \quad (3)$$

(1) Initially, the embedding vector $(\mathbf{x}_t)$ is multiplied by weights $(\mathbf{W}_q, \mathbf{W}_v, \mathbf{W}_k)$ to transform into query, key, and value vector (each $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t$). Following this, (2) the query and key vectors create correlation scores (s_{ti}) through scaled dot-product. Note that in autoregressive models, the positions for the keys precede the query.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3655953>

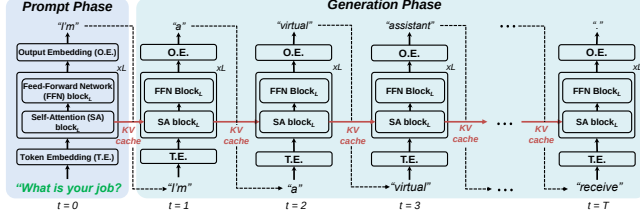


Figure 1: Transformer-based autoregressive text generation.

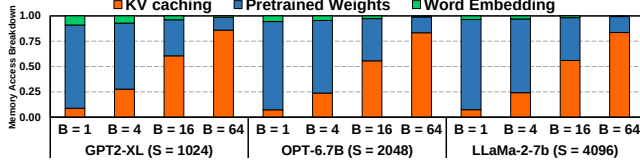


Figure 2: Memory transfer breakdown.

After that, the scores ($s_{t1}, \dots, s_{tt}$) are normalized into attention probabilities ($p_{t1}, \dots, p_{tt}$) using the softmax function, indicating strength of association between token x_t and x_i . (3) Finally, using these probabilities as weights, value vectors are multiplied and summed to produce attention output ($\mathbf{o}_t$). Subsequent operations include FFN layer, layer normalization, and residual connection to finalize one layer. Multiple iterations through subsequent layers followed by the final output embedding yield one token generation.

2.1.2 KV caching. As illustrated in Fig. 1, the text generation process consists of the prompt phase and generation phase. During the prompt phase, an entire prompt sequence ($x_1, \dots, x_n$) is used to predict a new token x_{n+1} (e.g., "I'm"). Within this phase, n sets of query, key, and value are produced and utilized for the self-attention block of each layer. Meanwhile, the generated keys ($\mathbf{k}_1, \dots, \mathbf{k}_n$) and values ($\mathbf{v}_1, \dots, \mathbf{v}_n$) are stored to prevent redundant creation in the following token generation. This technique is known as *KV caching*.

In the generation phase, tokens are sequentially generated until the maximum sequence length or an end-of-sequence token ($\langle \text{eos} \rangle$) is encountered. At a given time t , the model takes an input token x_{n+t} to produce the following token x_{n+t+1} . The input is constructed as a vector from a single token, leading to the execution of a General Matrix-Vector Multiplication (GEMV) operation that makes the workload memory-bound. Therefore, latency and energy consumption are dominated by off-chip access in this phase.

2.2 Motivation

2.2.1 Memory transfer overhead. Efficient batching technique was developed to improve the generation process. It spreads out the cost of loading pre-trained weights across multiple requests, thereby increasing inference efficiency. Yet, the unique *KV* cache in each self-attention cannot be shared, resulting in increased memory transfer overheads. Fig. 2 shows the breakdown of off-chip memory accesses during the generation phase for different batch sizes, all set to each model's maximum context length. While *KV* caching transfer is 7.8% at a batch size of 1, it becomes 84.3% for a batch size of 64, leading to prolonged generation latency as demonstrated in [9]. Thus, minimizing latency in self-attention is essential to handle larger batch sizes for enhancing the benefits from weight sharing. This emphasizes the need to reduce off-chip accesses for *KV* transfers in self-attention.

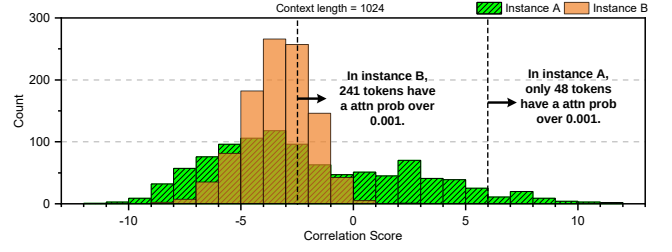


Figure 3: Various attention score distribution.

2.2.2 Distribution-aligned pruning. The softmax operation in self-attention amplifies differences among correlation scores exponentially, resulting in numerous tokens with near-zero p_i in Eq. (3). Thus, the corresponding value vectors $\mathbf{v}_i$ barely affect the attention output $\mathbf{o}_t$ and can be pruned with minimal performance loss. Using this property, there have been several works [1, 8, 15] to lighten the cost of self-attention block. Among those, SpAtten [1] reduced the access for *KV* cache with cascade token/head pruning and local value pruning, which retains tokens with the highest probability at a pre-defined ratio. While the method is effective in reducing *KV* transfers, it often overlooked variations in the number of unimportant tokens across instances.

Fig. 3 demonstrates the variability by comparing the number of dominant tokens (i.e., probability over 10^{-3}) in identical setups—same layer, head, and context length. In the two instances, only 4.6% of tokens in instance A exceed a probability of 10^{-3} , while 23.5% in instance B. This discrepancy arises from the relative nature of the softmax; wider distribution of scores, with greater differences between scores, leads to fewer dominant tokens. Therefore, fixed-ratio pruning strategy could either remove dominant or keep unimportant tokens, undermining the overall performance and efficiency. Furthermore, it necessitates additional fine-tuning steps for each dataset to attain a higher pruning ratio, resulting in significant costs for large language models. Other studies [8, 15] have also proposed token pruning methods, showing decent performance gains in bidirectional language models [7, 14]. However, they are not optimized for the memory-bounded generation phase as they require loading all *KV* pairs on-chip. Thus, previous methods become suboptimal in generation as they neither adjust the pruning method for individual instances nor reduce *KV* transfers.

3 PROPOSED WORK

To counteract the described variability, removing only the tokens with low probability is practical instead of removing them under fixed ratio. However, since the probability is determined by the difference between scores, it requires all correlation scores, which hinders the reduction of *K* transfers. To identify negligible tokens and reduce *K* transfers simultaneously, we propose a method that estimates the probabilities prior to completing the correlation score calculations.

Specifically, this method conservatively estimates the probability using the segmented bits (bit chunks) of *K*, and dynamically prunes tokens with the estimated value falling below a set threshold, thr (Sec. 3.1). If the pruning occurs before requesting the final chunk, the remaining *K* bit chunks and *V* vector transfer can be avoided; otherwise, the token is considered pivotal and the subsequent chunk is requested for more precise pruning decisions. In this procedure,

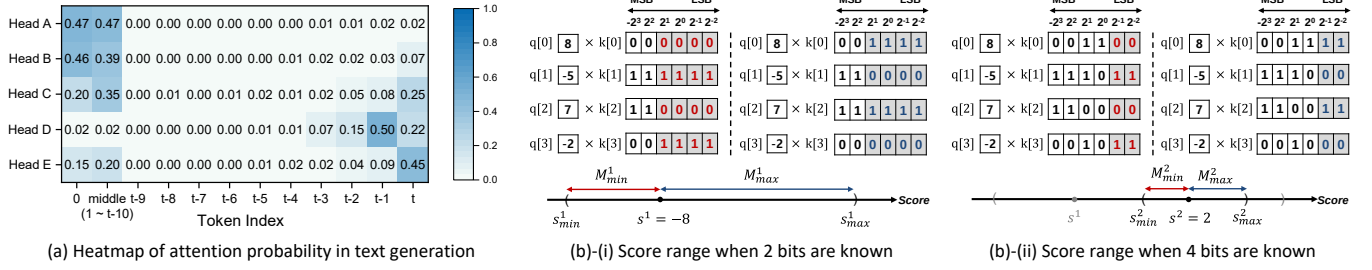


Figure 4: (a) Heatmap of attention probability across token indices in text generation, where the middle column aggregates probabilities for tokens from 1 to t-10. (b) Margins from partial score where true result exist. s^b indicates partial score of chunk index b . M_{min}^b and M_{max}^b imply margins for the minimum and maximum values, respectively.

there are two main challenges: 1) Errors by estimation lead to incorrect decisions (i.e. a pruned token has a probability larger than thr), potentially damaging accuracy (Sec. 3.1). 2) On-demand DRAM requests for bit chunks, unlike on-chip memory accesses, incur significant latency. This leads to the under-utilization of compute units (Sec. 3.2).

3.1 Probability Estimation

The nature of the exponent allows estimating probability p'_i as $\frac{\exp(s_i)}{\sum_{j \in subset} \exp(s_j)}$ when only a *subset* of tokens is known. Since the result of exponentiation is always positive, the inequality $p_i < p'_i$ holds. Therefore, if an estimated probability p'_i is below a predefined thr , the actual probability p_i will also be below, irrespective of any forthcoming scores for tokens $j \notin subset$.

To find negligible tokens as early as possible, it is advantageous to prioritize dominant tokens within the subset. To this end, we exploit the locality of attention mechanism observed in text generation. As shown in Fig. 4a, recently generated tokens and the first token often carry more weights than others. Therefore, beginning the score calculation with these tokens and progressing in reverse chronological order effectively enhances the pruning ratio.

To further reduce K transfers, we introduce a method that estimates the probability of a token using bit chunks of K . This method extends the *conservative margin* concept [15] for 2's complement number format. The margin represents the potential amount of change. For an N -bit integer $a_{N-1}a_{N-2} \dots a_0$, its value w is:

$$w = -a_{N-1}2^{N-1} + \sum_{i=0}^{N-2} a_i 2^i \quad (4)$$

In this format, all bits except the sign bit contribute a value of positive or zero. In a dot-product of two vectors, we can predict possible range of the result even if only a portion of bits of one operand is given. Fig. 4b details this concept. In this example, while Q retains all the bits, K has a fraction of the 6 bits: 2 bits in Fig. 4b-(i) and 4 bits in Fig. 4b-(ii). For elements of Q that are positive, setting the unknown bits of K , shown in gray, to 1 (or 0 if negative) yields potential maximum score s_{max}^b since it considers only increments. Conversely, flipping the unknown bits determines the minimum score s_{min}^b . Note that the margin pairs for each chunk index are determined solely by the Q vector. Using this concept, in expression $\frac{\exp(s_i)}{\sum_{j \in subset} \exp(s_j)}$, replacing the correlation score of numerator with s_{max}^b and the score of denominator with s_{min}^b yields maximally

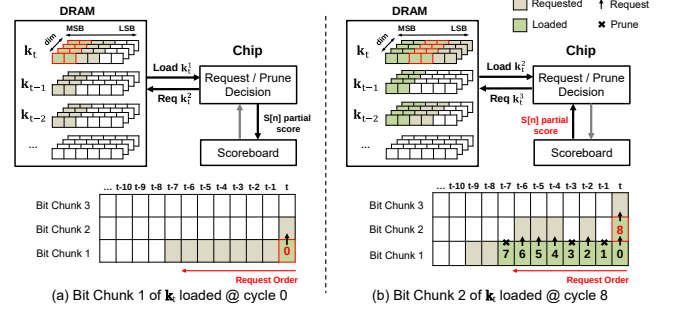


Figure 5: Out-of-Order Score Calculation

estimated probability p''_i by following relation:

$$\frac{\exp(s_i)}{\sum_{j \in subset} \exp(s_j)} \leq \frac{\exp(s_{i,max}^b)}{\sum_{j \in subset} \exp(s_{j,min}^b)} = p''_i \quad (5)$$

Thus, if p''_i falls below the thr , it is inferred that the p_i is as well. This method ensures the safe elimination of KV transfers of tokens with low probabilities.

In conclusion, our approach provides a conservative probability estimate. It identifies redundant tokens based on their relative importance compared to the subset, thus ensuring that all necessary tokens are retained in each instance.

3.2 Out-of-order Score Calculation

Our estimation method permits pruning decisions that rely on partial scores, which are computed using chunks of K . When a token is not pruned, the following chunk of the K is requested to DRAM for a more precise estimation. However, waiting for each request whenever it occurs leads to under-utilization due to off-chip memory accesses latency. To mitigate this, we introduce an out-of-order computing strategy, depicted in Fig. 5, which processes as follows:

- (1) When the score computation starts, only the first chunks of K vectors are requested in sequence.
- (2) Once any chunk is loaded from the DRAM, the partial score of the chunk is computed, the probability is estimated, and the decision on pruning is made.
- (3) If not pruned, the next chunk of that K is requested. Concurrently, its partial score is stored in the Scoreboard (Fig. 5a). Otherwise, the process of requesting the first chunk continues.
- (4) When any downstream chunk is loaded from DRAM (Fig. 5b), it fetches the previous partial score from the Scoreboard, followed

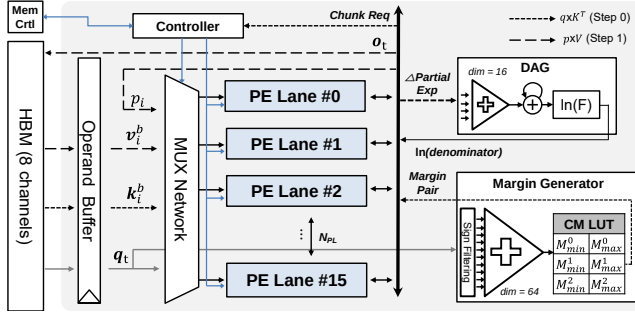


Figure 6: ToPick Overall Architecture

by updating new partial score. Then, it repeats the process (2) and (3) with the new score.

This out-of-order processing approach enables calculating the partial scores for different K s as soon as any chunk becomes available from DRAM. It facilitates the continuous score calculation through ongoing requests. For example, consider the period between the request (Fig. 5a) and the on-chip loading (Fig. 5b) of the second chunk of $\mathbf{k}_l$. During this interval, the first chunk of the other tokens is processed, which leads to either a request for a new first chunk or for the next chunk of non-pruned tokens. Therefore, this method keeps the Processing Element (PE) active and optimizes on-demand DRAM access, leading to faster execution with fewer bit accesses for pruned tokens.

This process seamlessly supports on-demand access to chunks of K , resulting in both speedup and reduced access. In the end, only the tokens that have not been removed by the last chunk participate in subsequent softmax and $\times V$ operations.

4 TOPICK ARCHITECTURE

This section presents the ToPick architecture (illustrated in Fig. 6), which implements self-attention mechanism with dedicated modules: the Denominator Aggregation Module (DAG) and the Margin Generator, both supporting probability estimation and out-of-order score calculation. Also, we design a lane-based processing element (PE Lane) for ToPick, where K/V vectors from the DRAM are partitioned to each PE Lane. The operand precision for self-attention is set to 12 bits, segmented into three 4-bit chunks for each K/V vector.

ToPick performs two main operations: $q_i^T \cdot k_i$ (step 0) and $\sum p_i v_i$ (step 1), where i indicates the index of a token. The MUX network in Fig. 6 configures the datapath for each stage, enabling dot-product calculations for step 0 and accumulative operations for step 1. During the prompt phase, all K/V vectors are preloaded into the on-chip buffer to be reused across queries. Conversely, in the generation phase, the q_i resides in the operand buffer and each KV chunk is streamed from DRAM, resulting in the execution of 12×4 bit operations. Following are the details about hardware modules utilized for probability estimation in the generation phase:

(1) Before starting step 0, the Margin Generator produces three margin pairs (M_{min}^b, M_{max}^b) for each chunk index b solely from the query. These margin pairs are then utilized during step 0 through a Look-Up Table (LUT) to support probability estimation.

(2) Equipped with 64 multipliers and an adder tree, PE Lanes perform dot-products and probability estimation with out-of-order execution to minimize data transfer in step 0. Following this, in

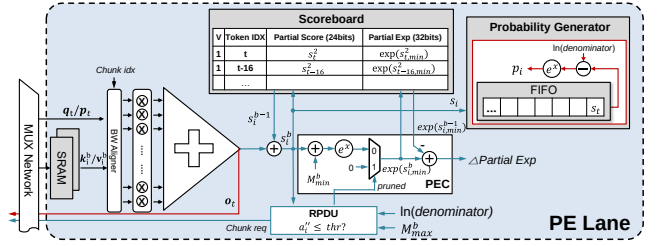


Figure 7: PE Lane Microarchitecture

step 1, they compute attention probabilities and request $\mathbf{v}_i$ for the unpruned tokens to make attention output $\mathbf{o}_t$. Further details on the PE Lane design for reducing data transfer will be provided in the subsequent subsection.

(3) DAG determines a real-time denominator of p'' . During each cycle, the differences between chunk indices ($\exp(s_{min,i}^b) - \exp(s_{min,i}^{b-1})$) from PE Lanes are aggregated to update the denominator, $\sum_{j \in subset} \exp(s_{j,min}^b)$. The natural logarithm of the denominator is distributed to all PE Lanes, facilitating the evaluation of $s_{i,max}^b - \ln(denominator) \leq \ln(thr)$, which is equivalent to $p'' \leq thr$. Following step 0, the denominator represents the exponentiated sum of the unpruned scores, which is utilized for the softmax operation.

4.1 Microarchitecture of PE Lane

Fig. 7 illustrates the PE Lane microarchitecture. It consists of four components, including the multiplier-adder tree and three modules supporting the probability estimation in out-of-order processing: (1) Scoreboard in each lane acts as temporary storage for buffering the partial results. The results include partial score s_i^b and the partial exp, the exponent value of $s_{i,min}^b$ (i.e., $s_i^b + M_{min}^b$), of unpruned tokens. (2) Request/Prune Decision Unit (RPDU) makes a prune decision and decides which chunk to request. (3) Partial Exp Calculator (PEC) makes the partial exp to aggregate the denominator. If the chunk is downstream one, calculates the difference of partial exp between chunks.

These modules jointly operates to make prune decision and creating the partial exp for aggregating the denominator in step 0. At first, the multipliers and adder-tree computes the dot-product result ps_i^b from a 12-bit vector $\mathbf{q}_t$ and a 4-bit chunk vector $\mathbf{k}_i^b$. At the same time, the Scoreboard is accessed with token index i . If previous chunk exist, the previous score s_i^{b-1} is fetched and updated, generating new partial score s_i^b .

After that, prune decision is determined in RPDU. The RPDU gets the upper margin $M_{max}^b \cdot \ln(denominator)$, and the partial score s_i^b . Then the unit determines $p_i'' \leq thr$ holds. If true, the unit requests new first chunk to DRAM. Otherwise, it requests the subsequent chunk vector for the $\mathbf{k}_i$ and store the partial results in Scoreboard.

PEC makes the partial exp that is $\exp(s_{i,min}^b)$. It generates a difference of exponent values between the chunk index to deduct the previous value $\exp(s_{i,min}^{b-1})$ from the denominator. All the difference values collected from PE Lanes are aggregated in the DAG.

The Probability Generator computes attention probabilities for unpruned tokens. After step 0, the FIFO buffers indices and scores of these tokens. It calculates the probability $\exp(s_i - \ln(\text{denominator}))$ that is equivalent to p_i . Simultaneously, it requests the corresponding $\mathbf{v}_i$ vector from DRAM. The calculated probabilities, p_i , are then

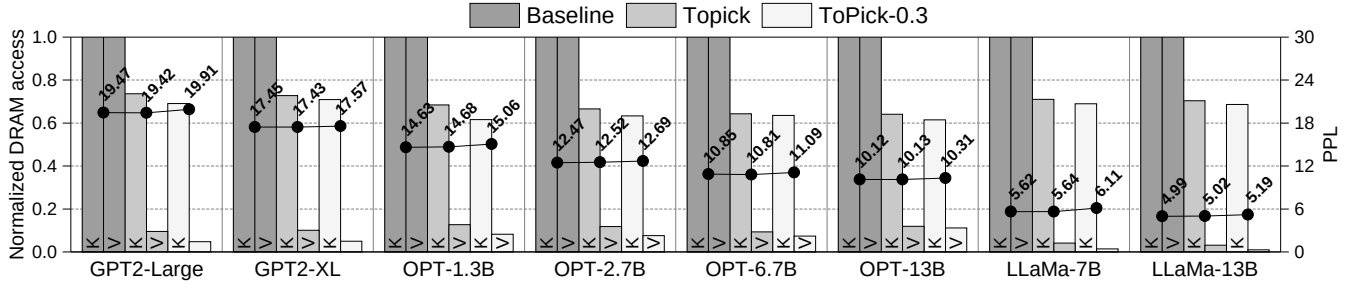


Figure 8: Required off-chip memory access in generation phase (bars) and algorithm performance (lines) across varying models.

forwarded to the PE lane, where the multiplier-adder tree performs the weighted sum $\sum p_i v_i$, forming the attention output $\mathbf{o}_t$.

5 EXPERIMENTS

5.1 Experimental Setup

5.1.1 Algorithm Evaluation Setup. We evaluate our proposed method on various language models tailored for text generation: GPT2-Large/XL [4], OPT-1.3B/2.7B/6.7B/13B [11], and LLaMa-2-7B/13B [6]. To analyze the impact of our methods on model performance, we measure perplexity (PPL) on the Wikitext-2 dataset [10], where lower perplexity values indicate better performance. This assessment utilizes pre-trained models available on HuggingFace [12].

5.1.2 Hardware Evaluation Details. Table 1 shows the hardware configuration of the ToPick architecture. We set the number of PE Lanes to 16 to fully utilize DRAM (HBM2) bandwidth in the generation phase, where each Lane processes a chunk (4 bits) of a K vector per cycle. We implement ToPick in RTL and synthesize it using Synopsys Design Compiler under Samsung 65nm LP standard cell library to evaluate the area and power consumption of ToPick at a target frequency of 500MHz (Table 2). We also use CACTI [2] to estimate the energy and area of on-chip buffers and scoreboard. To get the number of cycle and energy of off-chip accesses, we use DRAMsim3 [3] simulator with trace files generated in RTL simulation.

Table 1: Hardware Configurations of ToPick

Main Memory	HBM2; 8 channels $\times$ 128-bit at 2GHz; each channel provides 32GB/s bandwidth.
On-chip Buffer	192KB SRAM for each Key, Value buffer; 512B Operand buffer.
PE Lane	64-dim $\times$ 12-12 bit multipliers and adder tree; 32 entry $\times$ 67 bit Scoreboard; 2 $\times$ 32 bit fixed-point EXP unit.

5.1.3 Design Configurations. To evaluate the efficacy of our proposed method in the generation phase, we compared the proposed design to a baseline accelerator that lacks five hardware modules: Margin Generator, DAG, PEC, Scoreboard, and RPDU, which are integral to proposed optimizations. We evaluate two configurations with the baseline, ToPick and ToPick-0.3. The ToPick configuration includes modules supporting our methods, showing a minimal performance decrease of at most +0.05 PPL. In contrast, ToPick-0.3 is a configuration designed to balance hardware benefits with a slight performance decrease, allowing for increase of +0.3 PPL on average in Wikitext-2. For all hardware evaluations, we use context length of 1024 for GPT2 models and 2048 for OPT, LLaMa-2 models.

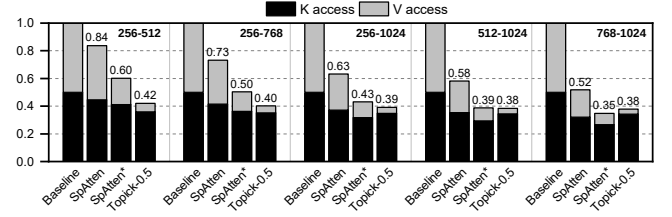


Figure 9: Normalized memory access comparison. ^{1 2}

5.2 Result

5.2.1 Memory access reduction. Fig.8 illustrates the impact of our method on the reduction of off-chip access for KV caching during the generation phase, showing a normalized comparison to the baseline. Our probability estimation scheme selectively prunes only the unimportant tokens in each instance, achieving a 12.1 $\times$ and 22.2 $\times$ reduction in V access in ToPick and ToPick-0.3 configurations, respectively. Moreover, the out-of-order score calculation method effectively eliminates the need to load remaining chunk of K for pruned tokens, resulting in a reduction of K accesses by 1.45 $\times$ and 1.51 $\times$ on average. As a result, our method achieves a total of 2.57 $\times$ and 2.79 $\times$ off-chip memory access reduction in each configuration.

Fig. 9 compares of ToPick-0.5 and Spatten [1] across various context lengths using the GPT2-Medium [4] model. For a fair comparison, we set the precision of Q, K, V to 12 bits and allow a +0.5 PPL in the Wikitext-2 dataset for both configurations. As shown in the figure, our scheme generally shows a better reduction ratio than SpAtten, except in the case of longer prompt setting where the cascaded token/head pruning method significantly reduces K access. However, when evaluating both designs without fine-tuning, our scheme shows a 1.64 $\times$ higher reduction in memory accesses compared to SpAtten.

5.2.2 Speed up & Energy Efficiency in Generation Phase. Fig. 10 presents the (a) speedup and (b) normalized energy breakdown of our ToPick accelerator during the generation phase for various models. As outlined in Sec. 2.1.2, the workload of the generation phase is memory-bounded; in the baseline design, latency and energy consumption are primarily due to off-chip memory accesses.

The introduction of probability estimation substantially reduces the access to V , yielding a speedup of 1.73 $\times$ and energy savings of 1.78 $\times$ compared to baseline. These results are due to the precise prediction of low probability tokens. Integrating out-of-order score

¹SpAtten* performs additional fine-tuning on Wikitext-2 dataset.

²The notation "a-b" for each cell indicates the prompt length and the ending length in text generation.

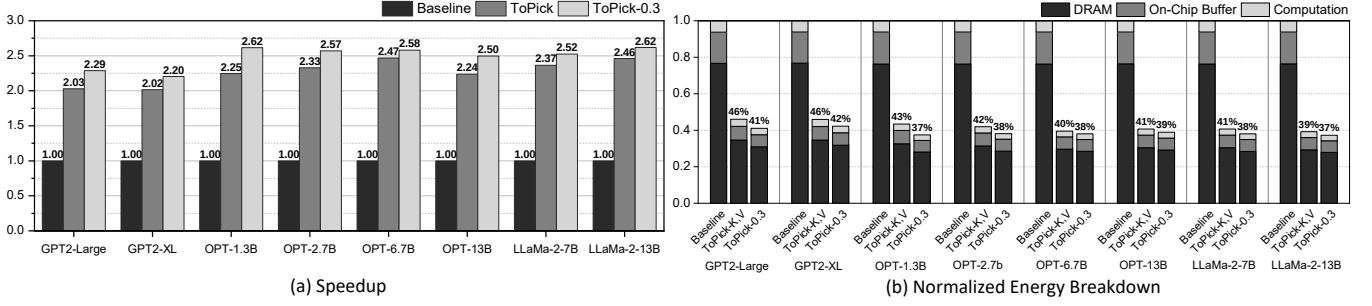


Figure 10: (a) Normalized energy breakdown and (b) Speedup of ToPick configurations in generation phase.

calculation, which is ToPick, leads to further benefits—a $1.32\times$ increase in speed and $1.35\times$ saving in energy consumption on average. The out-of-order technique hides the latency of on-demand DRAM access via independent score computations, thus accelerating the process. Moreover, by accepting a minor algorithmic degradation, ToPick-0.3 achieves a speedup of $2.48\times$ and energy efficiency of $2.63\times$. These results underscore the effectiveness of our proposed design in optimizing self-attention execution for text generation.

Table 2: Area and Power breakdown of ToPick at 500MHz.

Hardware Module		Area (mm ²)	Power (mW)
PE Lane $\times$ 16		2.518	426.76
PE Lane	Multipliers & Adder-Tree 12b	0.095	17.94
	Prob Gen	0.032	2.22
	PEC	0.004	0.73
	Scoreboard	0.024	4.69
	RPDU	0.001	0.17
Mux Network		0.076	3.13
Margin Generator		0.014	3.78
DAG		0.010	2.49
On-chip buffer		5.968	1053.32
Total		8.593	1492.78

5.2.3 Area & Power Analysis. Table 2 presents the area and power analysis for the ToPick accelerator. The additional modules, Margin Generator, DAG, and PEC, are designed to minimize V access on top of the baseline configuration, resulting in an area overhead of 1.0% and a power overhead of 1.3%. The ToPick further incorporates modules, Scoreboard and RPDU, aimed at reducing K access. This results in an additional area and power overhead of 4.9% and 5.6%, respectively, over the baseline. Nevertheless, the increase in power consumption is alleviated by significant power savings from the reduction of off-chip memory accesses. Therefore, the ToPick architecture demonstrates a strategic compromise, exchanging a moderate rise in power for a substantial improvement in energy efficiency.

6 CONCLUSION

In this work, we address the KV caching transfer overhead of self-attention in text generation. Firstly, we propose a probability estimation that identifies unimportant tokens using partial bits of K . This method prunes negligible words aligned with each instance, achieving a substantial pruning ratio for V without fine-tuning,

while also creating an opportunity to reduce K access. Secondly, we design an architecture that supports seamless on-demand off-chip DRAM access. Through out-of-order execution, our design avoids under-utilization by ongoing DRAM request. In conclusion, our proposed method reduces $2.57\times$ off-chip DRAM access for self-attention in text-generation, realizing a speedup of $2.28\times$ and $2.41\times$ energy efficiency.

7 ACKNOWLEDGMENTS

This research was supported by the MSIT (Ministry of Science and ICT), Korea, under the ITRC (Information Technology Research Center) support program (IITP-2020-0-01847) supervised by the IITP (Institute for Information & Communications Technology Planning & Evaluation) and BK21 FOUR (Connected AI Education & Research Program for Industry and Society Innovation, KAIST EE, No. 4120200113769). The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

- [1] Hanrui Wang et al. 2021. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 97–110.
- [2] Rajeev Balasubramanian et al. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (jun 2017), 25 pages. <https://doi.org/10.1145/3085572>
- [3] Shang Li et al. 2020. DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator. *IEEE Computer Architecture Letters* 19, 2 (2020), 106–109. <https://doi.org/10.1109/LCA.2020.2973991>
- [4] Alec Radford et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [5] Gyeong-In Yu et al. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [6] Hugo Touvron et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [7] Jacob Devlin et al. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [8] Liqiang Lu et al. 2021. Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 977–991.
- [9] Reiner Pope et al. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5 (2023).
- [10] Stephen Merity et al. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
- [11] Susan Zhang et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
- [12] Thomas Wolf et al. 2020. HuggingFace’s Transformers: State-of-the-art Natural Language Processing. *arXiv:1910.03771 [cs.CL]*
- [13] Woosuk Kwon et al. 2023. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180* (2023).
- [14] Zhenzhong Lan et al. 2019. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942* (2019).
- [15] Zheng Li et al. 2022. Accelerating attention through gradient-based learned runtime pruning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 902–915.